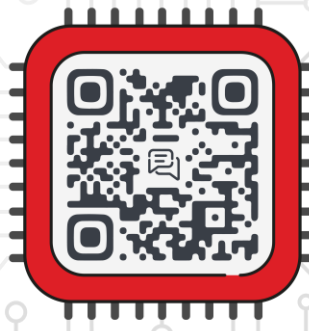




نظري

5

11

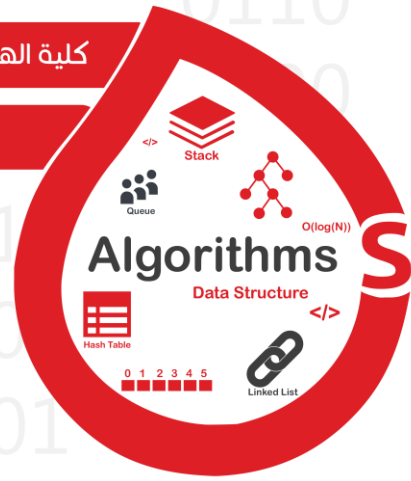
1320 sp

كلية الهندسة المعلوماتية

السنة الثانية

طرق التفكير 3

د. أبي صندوق



RB Informatics; 1/12/2024

خوارزميات 1

تعرفنا في المحاضرات السابقة عن طريقتي التفكير Divide and Conquer و Decrease and Conquer ونكمل اليوم مع عنوان Brute force .

ثالثاً: Brute force

هي طريقة لحل المشكلات عن طريق تجربة جميع الحلول الممكنة حتى الوصول إلى الحل الصحيح .
كل المشاكل التي تحل حاسوبياً حتماً لها حل Brute force قد يكون فعالاً أو لا . ومع ذلك إن توصلنا إليه وأجرينا بعض التعديلات فقد يصبح مفيداً .

مثلاً:

نريد ترتيب مصفوفة من 10 عناصر كيف نجرب جميع الحلول الممكنة هنا ؟

نقوم بتجربة جميع الترتيبات الممكنة

- فالعنصر الأول يمكن اختياره بـ 10 طرق
- والعنصر الثاني يمكن اختياره بـ 9 طرق
- العنصر الثالث يمكن اختياره بـ 8 طرق
- وهكذا.... ، أي لدينا 10! طريقة ممكنة علينا سردهم جميعاً حتى نصل للترتيب الصحيح

نلاحظ أن:

التعقيد هنا $n!$ أي أنها ستأخذ وقتاً طويلاً وبالتالي حل Brute force موجود ولكنه غير فعال .

▪ رأينا سابقاً أنه يوجد خوارزميات أفضل وأسرع بكثير مثل :

Merge Sort } $O(n \log n)$
Quick Sort

إذا استعملنا Merge Sort لترتيب الأقسام

insertion Sort } $O(n^2)$
Bubble Sort
Quick Sort

بفرض لدينا قناة تلفزيونية تعرض برامج متنوعة وبين البرنامج الأول والبرنامج الثاني لدينا فراغ 90 ثانية نريد أن نملأه بإعلانات مختلفة (بحيث لا نكرر الإعلان نفسه أكثر من مرة ونملأ أكبر قدر ممكن من الفراغ الموجود)

الإعلانات المتاحة :

رقم الإعلان	1	2	3	4	5	6
المدة الزمنية	40	35	30	25	22	19

الحل:

نقوم بتجربة كل المجموعات الجزئية من هذه المجموعة

أي تجربة مجموعة جزئية ($2^6 = 64$)

- ولا عنصر $\frac{6!}{0!6!} = 1$ +
 - كل عنصر لوحده $\frac{6!}{1!5!} = 6$ +
 - المجموعات الممكنة لعنصرين $\frac{6!}{2!4!} = 15$ +
 - المجموعات الممكنة لثلاثة عناصر $\frac{6!}{3!3!} = 20$ +
 - والمجموعات الممكنة لـ 6 عناصر $\frac{6!}{6!0!} = 1$ +
- 64

استعملنا التوافيق لحساب عدد المجموعات الممكنة

$$C = \frac{n!}{r!(n-r)!}$$

وذلك لأن الترتيب بين الإعلانات غير مهم .

▪ لنفرض أننا قررنا أن يكون الإعلان رقم 3 هو الإعلان الأول :

علينا تعبئة
90 ثانية .

1	2	3	4	5	6
40	35	30	25	22	19

فتصبح الإعلانات المتاحة

علينا تعبئة
60 ثانية .

1	2	4	5	6
40	35	25	22	19

- ولنفرض أننا قررنا أن يكون الإعلان الثاني هو رقم 1 :

فتصبح الإعلانات المتاحة

علينا تعبئة
20 ثانية .

2	4	5	6
35	25	22	19

× × × ✓

وبما أننا لا نستطيع تجاوز الـ 90 ثانية والإعلان رقم 2,4,5 لا يعتبر خياراً متاحاً ، لذا نختار الإعلان رقم 6 ونكون قد ملأنا 89 ثانية .

ولكن قد لا يكون هذا الجواب هو الأفضل نظراً لاختيارنا الإعلانات بشكل عشوائي ..

- لنجرب التراجع عن القرار الثاني وبدلاً من اختيار الإعلان رقم 1 سنختار الإعلان رقم 2 .

فتصبح الإعلانات المتاحة

علينا تعبئة
25 ثانية .

1	4	5	6
40	25	22	19

× ✓ ✓ ✓

ونكمل حتى نصل للجواب أو لطريق مسدود ...

وقد لا يكون أصلاً القرار الأول صحيح

نستنتج :

لإيجاد الإجابة الصحيحة علينا تجربه جميع الإجابات الممكنة:

- نقوم بتثبيت قرار ونكمل المسألة على أساسه.
- ثم نتراجع عنه ونختار قرار آخر ونكمل المسألة على أساسه.
- وهكذا حتى نجد الجواب الصحيح.
- وبذلك نضمن أننا جربنا كل الإجابات الممكنة .



The Best view comes
After , The Hardest climb.....

التعقيد:

تتم هذه الطريقة بتعقيد زمني $O(n!)$

حيث في البداية يكون لدينا 6 خيارات ثم 5 ثم 4 ثم 3 ثم 2 ثم 1 :

$$6 \times 5 \times 4 \times 3 \times 2 \times 1 = 6! \Rightarrow O(n!)$$

وهذا التعقيد مخفف عن التعقيد العام للـ Brute force في هذه المسألة ، حيث أنه في الحالة العامة يكون التكرار مسموحاً أي :

$$6 \times 6 \times 6 \times 6 \times 6 \times 6 = 6^6$$

و يكون التعقيد في الحالة العامة $O(n^n)$.

ولكن

إن أعدنا التأمل نلاحظ أنه أحياناً يتم عرض الدعايات نفسها ولكن بترتيب مختلف وفي مسألتنا لا يهمنا الترتيب أي هي مجرد حالات متكافئة حسابها غير مُجدٍ ومُكلف للوقت .

بالاعتماد على هذه الملاحظة :

يمكن اتباع آلية أخرى للتجريب :

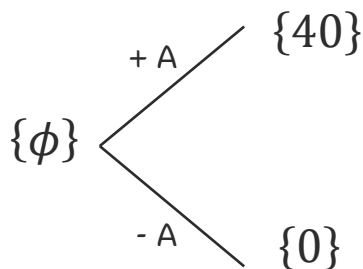
- بدلاً من أن نفكر (أي إعلان هو الأول بالترتيب) نفكر بكل إعلان هل سيعرض أم لا .
- إذ أن المطلوب هو مجموع الثواني لا ترتيب الإعلانات .

وبما أن كل إعلان سيكون أمام خيارين $\leftarrow$ يظهر $\checkmark$ لا يظهر $\times$

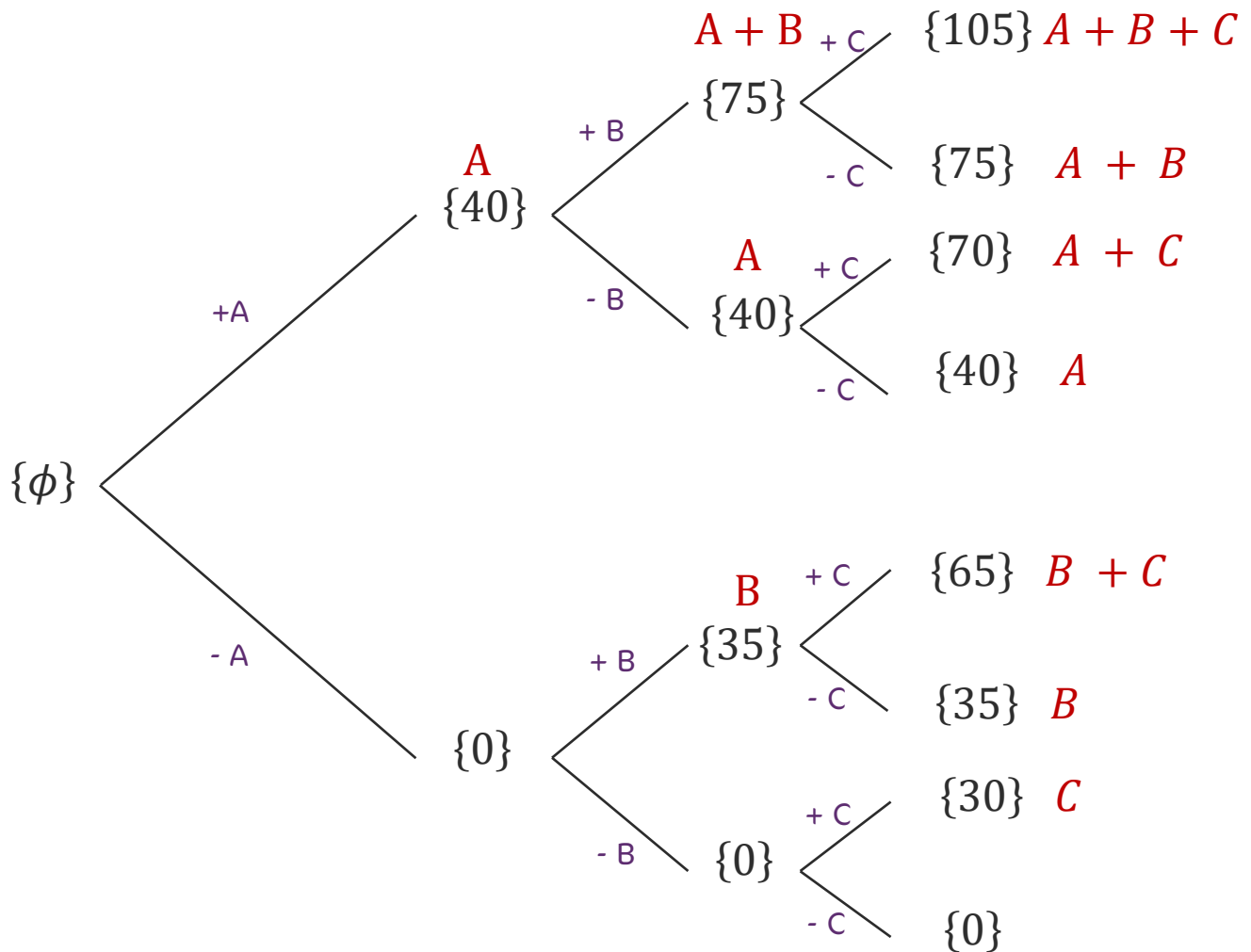
فالتعقيد الآن $O(2^n)$ أي 2^6 .

A	B	C	D	E	F
40	35	30	25	22	19

قرار بأخذ الإعلان A قلل المدة الزمنية من 90 إلى 50



قرار بعدم أخذ الإعلان A قلل الإعلانات من 6 إلى 5



وصلنا إلى 8 مجموعات .

وتستمر الشجرة حتى نصل إلى 64 مجموعة

ملاحظة :

- عندما وصلنا إلى المجموعة $A+B+C$ كنا قد ملأنا 105 ثانية أي تجاوزنا الـ 90 ثانية بالفعل . وبالتالي يمكننا قطع الفرع من هنا دون أن نكمّله لأنه يستحيل إيجاد مجموعة حلها صحيح من إكماله .
- إذاً في أسوأ الحالات ينتج لدينا مجموعة وقد ينتج أقل من 64 إذا تم قص بعض الفروع .

الخوارزمية :

شرح الكود:

السطر 1 : التابع الذي سنستدعيه للمرور على شجرة الاحتمالات.

السطر 2 : شرط التوقف أي إذا لم يبق خيارات (وصلنا لنهاية الفرع الذي نحن ضمنه) .

السطر 3 : التابع check optimum يختبر النتيجة التي حصلنا عليها بنهاية الفرع فإن كانت أفضل نتيجة يخرنها ، مثلاً: لو كانت آخر نتيجة مخزنه تغطي 80 ثانية:

- لو حصلنا بعدها على نتيجة تغطي 85 ثانية نحتفظ بها لأنها أفضل من 80 ثانية
- لو حصلنا بعدها على نتيجة تغطي 75 ثانية لا نحتفظ بها .

السطر 4 : وإلا (في حال وجود خيارات يمكن أخذها).

السطر 5 : من أجل كل قرار متاح.

السطر 6 : إذا كان قرار صالح (أي لم يتجاوز الـ 90 بعد).

السطر 7 : نقوم بحفظ الخيار (أي نعتبره جزءاً من الحل الحالي) .

السطر 8 : نعيد استدعاء التابع من أجل الخطوة التالية (أي من أجل الخيار التالي) .

السطر 9 : نزيل هذا القرار لنتخذ قراراً جديداً (أي نتراجع عن هذا القرار ونتخذ قرار جديد).

```

1. Try(int i){
2.   if(finalstate(i))
3.     check optimum();
4.   else{
5.     for each (option(v)){
6.       if(vaild option(v)){
7.         store option(v);
8.         Try(i+1);
9.         Backtrack option(v);
10.      }
11.    }
12.  }
13.}

```

ملاحظة:

■ Store Option(v) : أي بـ Data Structure ما .

1. خزن الإعلان A .

2. خزن عدد الثواني المتبقيه (50 هنا).

3. قم بإلغاء A من قائمة للخيارات المتاحة.

■ Backtrack Option(v) : أي اذهب لـ Data Structure نفسها .

1. قم بإلغاء تخزين الإعلان A .

2. أعد الثواني المحذوفة (عدنا إلى 90 ثانية هنا) .

3. أعد A إلى قائمة الخيارات المتاحة.

مثال عن Data Structure :

مصفوفة طولها بعدد الإعلانات

- إن أخذنا الإعلان نضع 1 .
- إن لم نأخذه نضع 0 .

الخوارزمية السابقة تمثل الشكل العام لـ Backtracking .

التعقيد:

وضوحاً التعقيد أسي

لكل إعلان لدينا خيارين أي $O(2^n)$.

لكن يمكن تخفيف التعقيد بالتلاعب بالشرط في السطر السادس بوضع شروط للاستمرار في الفرع .
عندها ننتقل من الـ Backtracking (والتي تعني تجريب كل خيار مهما كان) ليصبح لدينا نوع خاص يدعى:

• Branch & Bound

▪ Branch : فرع .

▪ Bound : قلم (أي قصّ الفروع غير الضرورية)

في السطر السادس في المثال السابق:

- إذا تجاوزنا الـ 90 ثانية فلا داعي للإكمال في الفرع .

ويمكن إضافة شرط آخر **مثلاً** :

- إذا كان مجموع الثواني من مكان وجودي بالفرع إلى نهايته لا يعطيني حلاً أفضل من حل موجود لدي سابقاً ، فلا داعي للإكمال في الفرع .

- إن تعقيد خوارزمية Backtracking كان $O(2^n)$.

أما تعقيد خوارزمية Branch & Bound فهو أخف من $O(2^n)$ ولكن يبقى ضمن الرتبة الأسية .

- نسبة التخفيف تعتمد على مدى وضوح قرارات الـ Bound وعددها .

في خوارزمتنا :

1. معامل التفرع (عند كل خيار كم فرع يتفرّع ؟) : 2 (a)

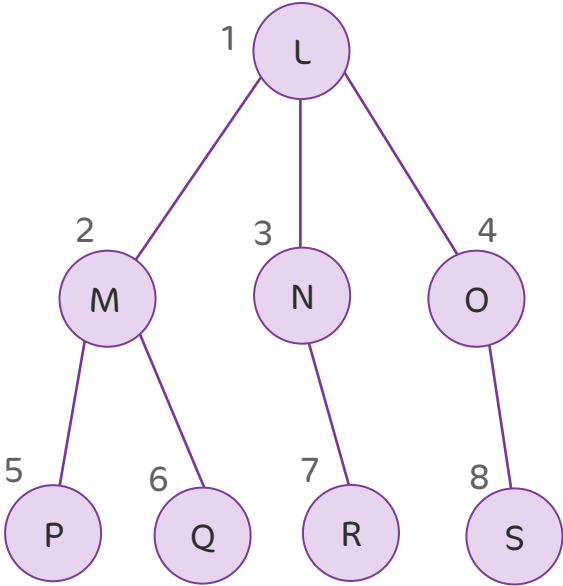
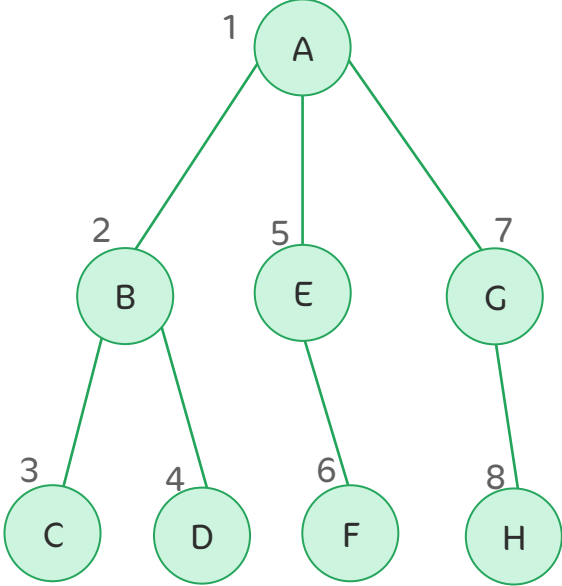
2. ما العمق المحتمل للشجرة ؟ كحد أقصى 6 (b)

3. عدد الخيارات الممكنة : كحد أقصى 2^6 (a^b)

التعقيد قلّ من 6^6 إلى 2^6 لأننا انتقلنا من خوارزمية من هي الدعاية الأولى فالثانية إلى خوارزمية هل ستظهر هذه الدعاية أم لا . وبهذا نكون قد قللنا الـ Breadth / Branch ويمكن تقليل الـ Depth عن طريق القصقة (Branch & Bound).

DFS & BFS

هما خوارزمتان لاستكشاف (أو المرور على) الـ Graphs أو الـ Trees .

BFS	DFS	الاسم
Breadth First Search بالعرض أولاً :	Depth First Search بالعمق أولاً :	التعريف
وهي تعتمد أن نسير في الأفرع معاً على التوازي .	وهي أن نأخذ الفرع و نولد جميع الاحتمالات الممكنة منه ، وأن نسير على هذا الفرع حتى نهايته ثم الانتقال إلى الفرع الآخر وهذا ما اعتمدناه في مسألتنا السابقة .	يعتمد في عمله على
ويعتمد على الـ Queue في عمله .	يعتمد على الـ Stack في عمله .	الخوارزمية
		النتيجة
	كلاهما يوصلان لنفس النتيجة لكن الـ DFS أسرع في التكويد لاعتماده على الـ Stack .	السرعة
	تختلف السرعة على حسب المسألة .	

تدل الأرقام على ترتيب الانتقال / التنفيذ

إثراء أربيسيزي :

↪ بعض المسائل التي تعتمد على خوارزمية Brute force :

المسألة	السبب
البحث في سلسلة غير مرتبة عن عنصر ما عن طريق التجوال عليها عنصراً عنصراً	لأن الطريقة تعتمد على تجريب كل الخيارات لإيجاد الخيار المطلوب .

ملاحظة :

مسألة سرد كل التباديل المحتمل لمجموعة من n عنصر ممكن أن تكون Brute force لكن الأصح أن تكون Decrease and conquer

↪ مسائل شهيرة نستعمل الـ Backtracking فيها :

اسم المسألة	المسألة	السبب
Subset Sum Problem	هل يمكن اختيار مجموعة فرعية في مجموعة أعداد بحيث يكون مجموعها مساوياً لقيمة محددة .	نحتاج لتجربة جميع المجموعات الفرعية والتراجع عند الوصول إلى مجموع غير صحيح
Maze Problem	العثور على طريق للخروج من متاهة معقدة	تجربة جميع التحركات المختلفة والتراجع عند الوصول لطريق المسدود
N-Queens Problem	وضع n ملكة على لوحة شطرنج بحيث لا تتعرض أي ملكة للاعتداء من الأخرى .	تجربة جميع الترتيبات الممكنة للملكات والتراجع عند وجود اعتداء بينهم .

↪ مثال أربيسيزي عن مسألة ال : Subset Sum Problem

```
#include <bits/stdc++.h>
using namespace std;
bool Try(int arr[], int sum, int n, int i, int total) { //Try(int i)
    if (i == n) { //if(finalstate(i))
        return total == sum; //check optimum();
    }
    else {
        if (total > sum) { //if(vaild option(v))
            return false;
        }
        if (total == sum) { //if(vaild option(v))
            return true; //store option(v);
        }
        total = total + arr[i];
        return Try(arr, sum, n, i + 1, total) || Try(arr, sum, n, i + 1, total - arr[i]);
        //the return (.....,i+1,...) is : Try(i+1);
        //the return (.....,total - arr [i]) is : Backtrack option(v);
        //the (total+arr[i] or total - arr[i]) is : for each (option(v))
    }
}

int main() {
    int arr[] = { 40, 35, 30, 25, 22, 19 };
    int sum = 300;
    int n = sizeof(arr) / sizeof(arr[0]);
    if (Try(arr, sum, n, 0, 0)) {
        cout << "TRUE";
    }
    else {
        cout << "FALSE";
    }
    return 0;
}
```

*THE END*



وإلى هنا يا أصدقاء، نكون قد أنهينا محاضرات مادة

الخوارزميات وبني المعطيات 1

نسأل الله تعالى أن نكون قد قدمنا أفضل ما لدينا ،

اعذرونا عن أي خطأ يظهر لكم

بالتوفيق للجميع ... ونلقاكم الفصل القادم إن شاء

الله

فريق الأربسيز يتمنى لكم كل التوفيق والنجاح

